

Quiz 1

Total Marks: 25

22 August 2025

1. [8 marks] State TRUE or FALSE for each of the below. Give a one or two sentence explanation about your answer. **Answers without justification will fetch zero marks, even if your answer is correct (1 mark each)**
- (a) If a formula is satisfiable, then it must be provable using the rules of Natural deduction, starting from an empty premise.
 - (b) SAT is NP-complete. So, given a witness, it is not always possible to check if the witness is a valid witness in polynomial time.
 - (c) The sequent $\varphi_1, \varphi_2, \dots, \varphi_n \vdash \psi$ has a proof iff $(\varphi_1 \wedge \varphi_2 \wedge \dots \varphi_n) \rightarrow \psi$ is a tautology.
 - (d) If we remove LEM from the set of proof rules for Natural Deduction, it becomes an *incomplete* proof system.
 - (e) Satisfiability checking of Horn formula is in P while validity checking of Horn formula cannot be done in polynomial time.
 - (f) Let A and B be two finite non-empty sets of wffs such that B is a strict subset of A (i.e, $B \subset A$). Let S_A and S_B denote the set of assignments which make all formulas in A and B true, respectively. Then, $S_A \subset S_B$ always.
 - (g) Every formula in propositional logic can be expressed as a conjunction of two formulae which are respectively in CNF and DNF, as well as expressed as a disjunction of two formulae which are respectively in CNF and DNF.
 - (h) A monotone-1-in-3 SAT formula is a formula in CNF, such that each clause has 3 positive literals. A 1-in-3 truth assignment is one where exactly one literal is set to true in each clause. The formula

$$(p \vee q \vee r) \wedge (p \vee q \vee s) \wedge (p \vee r \vee s) \wedge (q \vee s \vee r)$$

has a 1-in-3 truth assignment satisfying it.

Solution

- (a) **False.** This is true for a valid formula. If a formula, say φ can be derived from empty premise, then $\vdash \varphi \implies \models \varphi$. This is not true for any satisfiable, φ in general. (Choose $\varphi = p$ and clearly $\not\models \varphi$).
- (b) **False.** Since SAT is in NP, given a witness, we can always check if the witness is a valid witness in polynomial time. Coming up with the witness need not be possible in polynomial time.
- (c) **True.** We can easily construct a proof for $\varphi_1 \wedge \varphi_2 \wedge \dots \varphi_n \vdash \psi$ from that of $\varphi_1, \varphi_2, \dots \varphi_n \vdash \psi$ using $\wedge_e$ rule. Then we have $\varphi_1 \wedge \varphi_2 \wedge \dots \varphi_n \models \psi$ and thus, $\models (\varphi_1 \wedge \varphi_2 \wedge \dots \varphi_n) \rightarrow \psi$ showing that it is a tautology.
- (d) **False.** LEM can be derived from other rules in Natural deduction. So, removing LEM cannot affect the completeness.
- (e) **False.** Horn formulas are already in CNF and validity checking of CNF can be done in polynomial time.
- (f) **False.** There can be choices of A and B such that $S_A = S_B$. For example, choose $A = \{p, q, p \wedge q\}$ and $B = \{p, q\}$. Then it is easy to see that $S_A = S_B$. (Recall our discussion from Tutorial 3 on Question 8 :))
- (g) **True.** Let F be any propositional formula. Let F_1 be the formula in CNF equivalent to F and F_2 be the formula in DNF equivalent to F . Then, $F \equiv F_1 \wedge (p \vee \neg p)$ and $F \equiv (p \wedge \neg p) \vee F_2$ for some propositional variable, p .
- (h) **False.** In the first clause, if we make p true, we cannot make q, r, s true as q and r are together with p in first clause and s is together with p in second clause. Similarly, if we choose to make q true in the first clause, p (together with q in first clause), r (together with q in first clause) and s (together with q in second clause) cannot be made true. Similar argument goes for making r true. Thus this formula is not monotone-1-in-3 SAT

Rubrics

- 1 mark given for each sub part with clear and complete justification.
- 0.5 marks given if the justification has minor flaws but idea is clear.
- 0 marks given for following scenerios:
 - Wrong answer (i.e, True or False)
 - Answer is correct but no justification.
 - Totally wrong or irrelevant justification

2. [2 marks] Let F and G be formulas of propositional logic. Let H be the CNF formula

obtained by applying the CNF algorithm (as done in class) to the formula $F \wedge \neg G$. Then show that the following are equivalent:

- (a) $F \models G$
- (b) $F \vdash G$
- (c) $\emptyset \in \text{Res}^*(H)$

For this question, you are free to directly use the results proven in class without proving them again.

Solution

$F \models G \implies F \vdash G$ due to *completeness* of propositional logic.

$F \vdash G \implies F \models G$ (due to *soundness* of propositional logic) $\implies$ In any row of the truth table, if F is true, G is also true $\implies$ In any row of the truth table either F is false or $\neg G$ is false $\implies F \wedge \neg G$ is always false $\implies H$ is unsatisfiable $\implies \emptyset \in \text{Res}^*(H)$.

Now, if $\emptyset \in \text{Res}^*(H)$, H is unsatisfiable $\implies F \wedge \neg G$ is unsatisfiable $\implies$ In all the rows of the truth table where F is true, $\neg G$ is false (i.e, G is true) $\implies F \models G$

Rubrics

- For solutions proving $(a) \equiv (b)$ and $(b) \equiv (c)$ (or some variant of this)
 - 0.5 marks for using soundness and completeness to argue the first part
 - 1 mark for proving one equivalence and an implication in one direction for the other equivalence
 - 2 marks for proving both equivalences completely
 - 0.5 marks have been deducted in case minor mistakes were made in the justification
- For solutions proving $(a) \implies (b)$, $(b) \implies (c)$ and $(c) \implies (a)$
 - 0.5 marks for using soundness and/or completeness to argue about the equivalence of (a) and (b)
 - 1 mark for proving one equivalence and an implication in one direction for the other equivalence
 - 2 marks for proving all three implications correctly
 - 0.5 marks have been deducted in case minor mistakes were made in the justification

3. [5 marks] Use natural deduction to prove the following sequent:

$$p \vee q, \neg(p \wedge q) \vdash (p \wedge \neg q) \vee (q \wedge \neg p)$$

You must use only $\vee_i, \vee_e, \wedge_i, \wedge_e, \perp_i, \perp_e, \neg_i, \neg_e$ rules in your proof. You are **strictly not allowed** to use any other rules or LEM in your proof. Any proof that invokes additional rules or semantic arguments (e.g., truth tables, equivalences) will receive zero marks. Also, make sure that you give a **single complete proof** for this question. Proofs that rely on sub-proofs won't be evaluated. You must also annotate each step of your proof with the basic rule applied at that step.

Solution

1.	$p \vee q$	premise
2.	$\neg(p \wedge q)$	premise
3.	p	assumption
4.	q	assumption
5.	$p \wedge q$	$\wedge_i$ 3,4
6.	$\perp$	$\perp_i$ 2,5
7.	$\neg q$	$\neg_i$ 4-6
8.	$p \wedge \neg q$	$\wedge_i$ 3,8
9.	$(p \wedge \neg q) \vee (q \wedge \neg p)$	$\vee_i$ 8
10.	q	assumption
11.	p	assumption
12.	$p \wedge q$	$\wedge_i$ 10,11
13.	$\perp$	$\perp_i$ 2,12
14.	$\neg p$	$\neg_i$ 11-13
15.	$q \wedge \neg p$	$\wedge_i$ 10,14
16.	$(p \wedge \neg q) \vee (q \wedge \neg p)$	$\vee_i$ 15
17.	$(p \wedge \neg q) \vee (q \wedge \neg p)$	$\vee_e$ 1, 3-9,10-16

Rubrics

- 5 Marks: Fully correct solution
- 1 Mark deducted for each wrong/missed line of proof
- 0.5 Mark deducted for each misplaced/missing box
- Partial Marks (< 3) for incomplete proofs
- 0 marks for using rules out of the ones mentioned in the question or using rules incorrectly/conceptual errors

4. [5 marks] Consider the following procedure for resolution. Start with a formula F in CNF written in the set notation as done in class. For example, $F = (p \vee q) \wedge (\neg p \vee q \vee r)$ is written as $F = \{\{p, q\}, \{\neg p, q, r\}\}$.

Step 1 If there are no variables in F , check satisfiability of F by checking whether $\emptyset \in F$.

Step 2 Choose a variable p such that there is at least one pair of clauses in F which can be resolved using p .

Step 3 If no such variable exists, then we are done with resolution. Check satisfiability by checking whether $\emptyset \in F$.

Step 4 Let C^p be the set of clauses in F which can be resolved wrt p . Let $Res_p(F)$ denote the set of clauses obtained by resolving clauses in C^p and removing from F , the clauses involved in the resolution wrt p . Update F to be $Res_p(F)$.

Step 5 Go to Step 1.

Is the above algorithm a complete resolution procedure? That is, is it the case that F is unsatisfiable iff $\emptyset \in F$?

If you think the above algorithm is a complete resolution procedure, give a formal proof for the same. Otherwise, argue using a counter example why it is not the case.

Solution

To show that the algorithm works, we have to show that F is equi satisfiable with $Res_p(F)$. In particular, show that $F \vdash \perp$ iff $Res_p(F) \vdash \perp$.

One direction is easy, namely when $Res_p(F) \vdash \perp$. By the soundness of resolution, any clause derived by resolving two clauses is semantically/syntactically entailed by those clauses. Hence, $Res_p(F) \vdash \perp \implies F \vdash \perp$. [Note that this is equivalent to proving that F is satisfiable $\implies Res_p(F)$ is satisfiable.]

Let us now show the other direction. We prove that if $Res_p(F)$ is satisfiable, then F is satisfiable. We can write F as follows.

$$F = \{\{p\} \cup A_i \mid 1 \leq i \leq m\} \cup \{\{\neg p\} \cup B_j \mid 1 \leq j \leq n\} \cup \{C_k \mid 1 \leq k \leq \ell\}$$

such that A_i, B_j, C_k contain neither p nor $\neg p$. The first set is all clauses in F containing p , the second is all clauses in F containing $\neg p$, and the last set is all clauses of F which contain neither p nor $\neg p$.

$$Res_p(F) = \{A_i \cup B_j \mid 1 \leq i \leq m, 1 \leq j \leq n\} \cup \{C_k \mid 1 \leq k \leq \ell\}$$

such that no $A_i \cup B_j$ contains some q as well as $\neg q$ (so not a tautology).

Assume there is some assignment α such that $\alpha \models Res_p(F)$. Then $\alpha \models C_k$, $1 \leq k \leq \ell$. If $\alpha \models A_i$ for all i , then $\alpha[p \mapsto 0] \models F$. If $\alpha \not\models A_h$ for some h , then $\alpha \models A_h \cup B_j$ for all $1 \leq j \leq n$. This is so since $A_h \cup B_j \in Res_p(F)$ for $1 \leq j \leq n$ and $\alpha \models Res_p(F)$. Now since $\alpha \not\models A_h$, it must be that $\alpha \models B_j$ for $1 \leq j \leq n$. Then $\alpha[p \mapsto 1] \models F$ and F is satisfiable.

Alternately, it can be proved that if F is unsat, for any assignment, there is some clause in $Res_p(F)$ which is false, making $Res_p(F)$ also unsat.

Rubrics

The following answers get *zero* marks directly:

- If your answer is that the algorithm is not complete.
- If you just say that the algorithm is complete but does not give a formal proof for the same.

No cribs will be entertained if you fall in any of the either case above.

Detailed mark distribution.

- 1 mark for the easy direction, i.e, showing that one of the below two is true (by soundness of resolution):
 - $Res_p(F) \vdash \perp \implies F \vdash \perp$
 - F is satisfiable $\implies Res_p(F)$ is satisfiable.
- For the non-trivial direction, $[Res_p(F)$ is satisfiable $\implies F$ is satisfiable] (or equivalently) $[F$ is unsat $\implies Res_p(F)$ is unsat], marks are distributed as follows:
 - 4 marks for a completely correct proof.
 - 2 marks if the idea of assigning $p \mapsto 0$ and $p \mapsto 1$ in different satisfying assignments of $Res_p(F)$ is written, but the exact structure of $Res_p(F)$ (that it contains the resolvents from all pairs of clauses that could have been resolved with the variable p) is not written clearly.

Common Mistakes.

- Some people tried proving the forward direction saying that “If we get ϕ in the last step, then we must have got it by resolving $\{p\}$ and $\{\neg p\}$ for some variable p . Hence, $F \vdash p, F \vdash \neg p$ and therefore $F \vdash \perp$, that is, F is unsat.” This is incomplete, since it is not obvious that $F \vdash p$ and $F \vdash \neg p$. This only follows from the soundness of resolution steps taught in class. Only those who have mentioned that this follows from the “soundness of resolution” have been given that 1 mark.
- Many people have tried to use induction on the number of variables, and did it for just the base case, without proving the main crux of the problem (that F is equisatisfiable to $Res_p(F)$) which is required in the inductive step. Such solutions have not been awarded marks.

5. [5 marks] Let $G = (V_G, E_G)$ and $H = (V_H, E_H)$ be two simple undirected graphs with the same number of vertices $|V_G| = |V_H| = n$. The *Graph Isomorphism* problem asks whether there exists a bijection (permutation) $f : V_G \rightarrow V_H$ such that for every edge $(u, v) \in E_G$, the edge $(f(u), f(v)) \in E_H$.

Given two finite graphs G and H with n vertices, describe how to obtain a propositional formula φ_{GH} such that φ_{GH} is satisfiable iff G is isomorphic to H . φ_{GH} must be computable

in time polynomial in n .

Answers without proper explanation are bound to lose marks. Make sure you properly indicate what each propositional variable encodes.

Solution

Define variables p_{ij} as vertex $i \in V(G)$ maps to vertex $j \in V(H)$. Let $E_G(u, v) = \top$ iff $(u, v) \in E(G)$ and $\perp$ otherwise, and similar with H .

Function Constraint.

$$\phi_f = \bigwedge_{i \in V(G)} \left(\bigvee_{j \in V(H)} p_{ij} \wedge \bigwedge_{j, k \in V(H), j \neq k} \neg (p_{ij} \wedge p_{ik}) \right)$$

Bijection Constraint.

$$\phi_{bij} = \phi_f \wedge \bigwedge_{j \in V(H)} \left(\bigvee_{i \in V(G)} p_{ij} \wedge \bigwedge_{i, k \in V(G), i \neq k} \neg (p_{ij} \wedge p_{kj}) \right)$$

Edge Constraint.

$$\phi_e = \bigwedge_{u, v \in V(G)} \bigwedge_{u', v' \in V(H)} (p_{uu'} \wedge p_{vv'}) \rightarrow (E_G(u, v) \leftrightarrow E_H(u', v'))$$

Isomorphism.

$$\phi_{bij} \wedge \phi_e$$

Rubrics

- 1 Mark: Function Constraint
- 1 Marks: Bijection Constraint
- 2 Marks: Edge Constraint
- 1 Mark: The whole formula being correct, with some explanation of the encoding

PS: The wording in the question implies the subgraph problem and not the isomorphism problem. We have given the 2 marks for edge constraint assuming either of these